

RadarRun Pipeline AS IS

Current as of Slice 2.17.4.6.2.

This document is the factual runtime contract for the current RadarRun pipeline. It describes what the product does today, what evidence proves it, and which boundaries must be protected when future search, signal, and candidate slices change the system.

The broader upstream architecture map stays in `docs/architecture/UPSTREAM_SEARCH_AND_SIGNAL_ARCHITECTURE.md`. This file is the dedicated RadarRun pipeline AS IS source, symmetric to `DRAFT_RUN_PIPELINE_AS_IS.md`.

PDF quick view: `docs/architecture/RADAR_RUN_PIPELINE_AS_IS.pdf`.

Regenerate with:

```
python scripts/generate-draft-run-pipeline-pdf.py `
--source docs/architecture/RADAR_RUN_PIPELINE_AS_IS.md `
--output docs/architecture/RADAR_RUN_PIPELINE_AS_IS.pdf
```

How AS IS Participates in DoD

For any complex RadarRun, upstream search, source signal, scoring, benchmark, or candidate assembly slice, this document must be used twice:

1. Before implementation, as a requirement source for the slice DoD.
2. After implementation, as a validator for what changed or stayed invariant.

A DoD for a RadarRun slice must explicitly state:

- which AS IS invariants are preserved;
- which AS IS behavior is intentionally changed;
- which TO BE or ADR document authorizes the change;
- which runtime artifacts prove the result;
- whether this document and PDF must be updated after the slice.

Use the same lifecycle as other complex pipeline work: AS IS -> Change Intent -> TO BE -> DoD -> Implementation -> AS IS Update.

Required DoD evidence for RadarRun/search work comes from structured artifacts, not from subjective inspection:

- `searchPlan.strategy`, `language`, `intents`, `queries`, `sourceStrategy`, `skippedIntents`, and `skippedIntentDetails`;
- planned and executed coverage: `plannedCoverage`, `executedCoverage`, and `skippedRequiredCoverage`;
- operation timeline: provider query operations, skipped operations, warnings, and errors;
- `rawResults`, `selectedForRead`, `rejectedBeforeRead`, URL-read outcomes, and `foundMaterialIds`;
- `FoundMaterial` records stored in the project workspace;

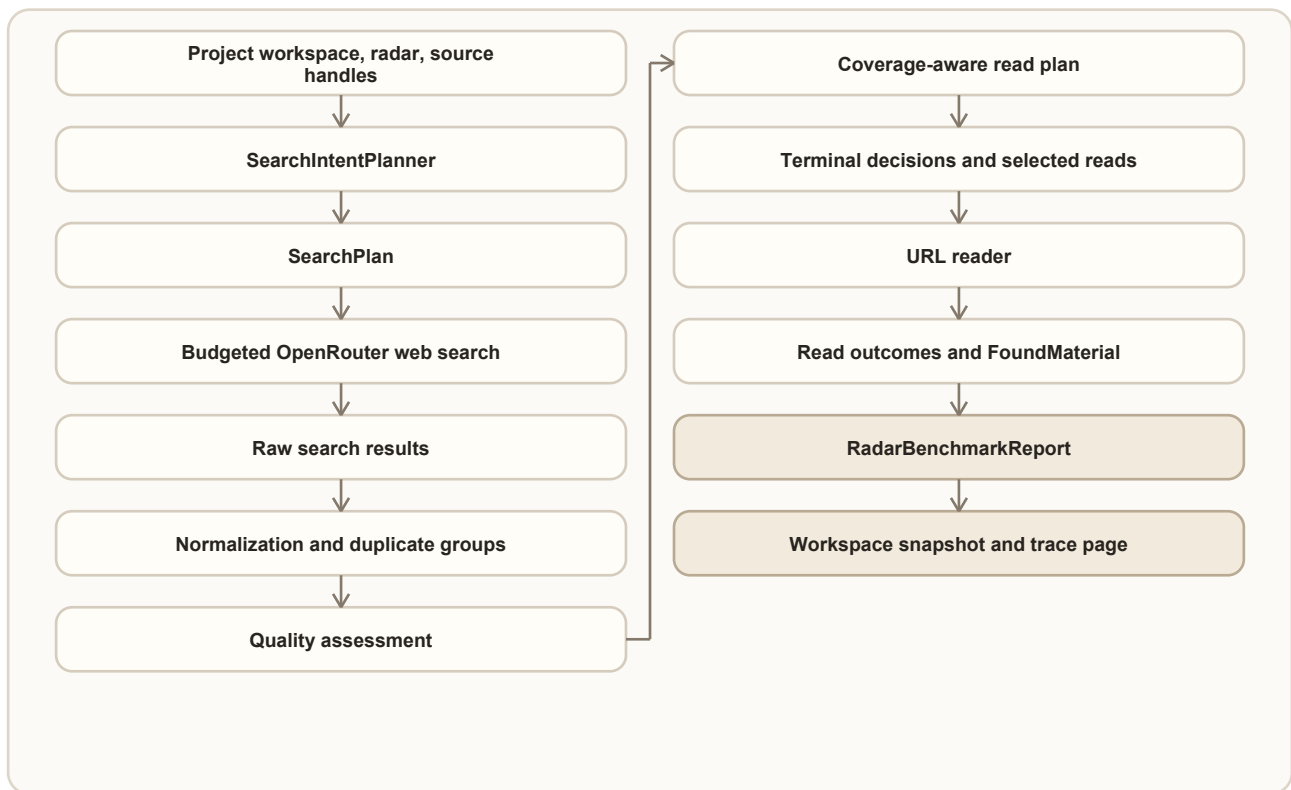
- benchmarkReport when a matching scenario exists;
- /radar-runs?runId=<id> trace page rendering for human diagnostics.

If a slice changes RadarRun behavior but does not update this AS IS contract or state why the contract remained valid, the slice is not ready to close.

Core Concepts

Concept	Current role	Does not own
SourceHandle	Project-scoped descriptor for a source that may be searchable, readable-only, paused, or needs review.	Provider execution, signal approval, candidate ranking.
RadarDefinition	User-facing radar settings: scope, source handles, trigger rules, filters, execution mode, and budget caps.	Search result storage or downstream drafting.
SearchPlan	Deterministic provider-free campaign plan with strategy, language, intents, queries, source strategy, budget, and skipped intents.	Provider calls or quality scoring.
SearchIntent	One planned evidence direction such as broad discovery, case/example, benchmark/paper, OSS/tooling, limitation/critique, or freshness.	URL reading or material acceptance.
SearchQuery	One provider-executable web-search query derived from an intent and eligible source strategy.	Search result quality judgment.
RadarRun	One execution attempt: status, budget usage, operations, search plan, raw results, read decisions, material ids, warnings, errors, and optional benchmark report.	SourceSignal, PostCandidate, plan slot, or DraftRun creation.
RadarRunOperation	One provider/search/read operation with status and trace-safe payload.	Editorial approval.
RawSearchResult	Normalized provider search result with query/run provenance.	Durable source memory or signal ownership.
SearchTriageReport	Deterministic normalization, duplicate groups, six-dimensional quality assessment, read plan, coverage gaps, terminal decisions, and read outcomes.	Provider search, URL parsing, or editorial approval.
selectedForRead	Search results chosen for URL reading within budget.	Proof that the material is accepted.
rejectedBeforeRead	Results rejected before URL reading, including duplicates, budget skips, and noise.	Permanent deletion from future search memory.
FoundMaterial	Retrieval output with provenance, title, URL/source ref, snippet or summary, warnings, and captured timestamp.	Topic/fabula ownership or final post candidate approval.
RadarBenchmarkReport	Recorded or live evaluation against a golden scenario.	Search execution or UI-side scoring.
RadarRunTracePage	Frontend read model for inspecting one run.	Recomputing live quality or mutating the run.

Runtime Topology



The current implementation stores RadarRun and FoundMaterial data in the workspace snapshot. There is no dedicated RadarRun SQLite table and no separate HTTP trace endpoint in this AS IS state.

Current Step Order

1. Resolve project, workspace, radar, source handles, language, research depth, and execution mode.
2. Classify source handles by eligibility:
 - searchable;
 - readable-only;
 - paused;
 - needs review;
 - unavailable or unsupported.
3. Build a deterministic SearchPlan:
 - source strategy;
 - campaign trace;
 - planned intents;
 - query families;
 - evidence types;
 - budget caps;
 - skipped intents and reasons.
4. Apply maxExternalQueries and produce provider-executable queries[].
5. Apply the direct openWebQuery input budget and final serialized-message guard.
6. Run provider web-search operations for executable queries and record provider usage

when OpenRouter returns it.

7. Normalize and bound provider citations into `rawResults[]` with `queryId` provenance.
8. Build stable duplicate groups that retain every query, intent, family, and evidence handle.
9. Assess representatives by relevance, evidence fit, project fit, source quality, novelty, and noise risk.
10. Build a coverage-aware read plan within the active 1/2/4 read cap.
11. Give every raw result exactly one terminal decision: selected, rejected, duplicate, invalid, or deferred by budget.
12. Read selected URLs when the format is supported and URL reading is available.
13. Record each read outcome. Successful text becomes a readable `FoundMaterial`; failed or unsupported reads become `metadataOnly` and do not count as readable.
14. Attach `benchmarkReport` when the run matches a golden scenario.
15. Persist the updated workspace snapshot.
16. Render compact radar trace and, when opened, the dedicated `/radar-runs` trace page.

Context Handoff and Execution Contract

RadarRun does not pass hidden state between roles. The handoff is persisted in the workspace snapshot and in the run payload.

Stage	Input artifacts	Output artifacts	Required proof
Source eligibility	Project workspace, RadarDefinition, source handles	<code>searchPlan.sourceStrategy</code> , skipped source reasons	Trace shows searchable, readable-only, paused, and needs-review handles.
Campaign planning	Radar scope, language, topics/fabulas as context, publisher/editorial rules, source strategy, budget mode	<code>searchPlan.intents</code> , queries, skippedIntentDetails, campaign trace	Planned intents and skipped reasons are visible in <code>searchPlan</code> .
Query budgeting	Planned intents, <code>maxExternalQueries</code>	Bounded <code>queries[]</code> , skipped intent reasons	Required directions skipped by budget appear as skipped coverage.
Provider search	Executable queries[], provider config, upstream budget profile	<code>RadarRunOperation</code> , provider citations, raw results	Direct providerInput, payloadBudget, messageCharCount, operation status, provider usage, errors, warnings, and provenance.
Triage and dedupe	Bounded <code>rawResults[]</code> , read budget, project/search context	<code>searchTriage</code> , selectedForRead, rejectedBeforeRead	Stable duplicate groups, dimension scores, one terminal decision per raw result, coverage, and gaps.
URL read	Selected reads, supported-format policy, URL reader adapter	read outcomes, readable or metadataOnly material	URL-read operation status, readable, failure reason, and material warnings.
Material output	Search/read payloads	<code>FoundMaterial</code> , <code>foundMaterialIds</code>	Workspace contains the material and run links it by id.
Benchmark evaluation	Scenario, run, found materials	<code>benchmarkReport</code>	Recorded/live status, provider health, coverage, missing expectations, and noise hits.

Hard Output Boundaries

RadarRun is a retrieval and trace pipeline. It may create FoundMaterial1.

It must not create:

- SourceSignal;
- PostCandidate;
- plan slots;
- DraftRun;
- final topic/fabula ownership for raw material.

Those downstream artifacts belong to separate review, scoring, candidate assembly, and planning slices. A RadarRun may expose affinity context, but it must not silently turn raw search results into approved editorial work.

Trace Contract

The current RadarRun trace contract is intentionally explicit. Future work must not collapse it into a single "results" blob.

Trace field	Meaning	Why it matters
searchPlan.strategy	Campaign strategy chosen by the deterministic planner.	Explains why this run searched in this shape.
searchPlan.language	Query language.	Prevents hidden RU/EN drift.
searchPlan.intents[]	Planned evidence directions.	Proves what the radar wanted to cover.
searchPlan.queries[]	Provider-executable queries.	Proves what could actually run under budget.
searchPlan.sourceStrategy	Source handle eligibility and use.	Explains searchable versus readable-only sources.
searchPlan.skippedIntents[] and skippedIntentDetails[]	Directions not executed and why.	Makes budget and source gaps visible.
operations[]	Provider and read operations.	Separates provider/runtime health from quality.
rawResults[]	Normalized provider citations.	Shows what the provider returned before triage.
searchTriage	Versioned candidates, scores, duplicate groups, read plan, coverage gaps, decisions, counts, and read outcomes.	Proves that no result disappeared and explains why each read slot was allocated.
selectedForRead[]	Raw results chosen for URL reading.	Shows read-budget choices.
rejectedBeforeRead[]	Raw results rejected before read.	Shows duplicates, noise, and budget skips.
operations[].providerInput and payloadBudget	Direct current-call provider input and limits.	Prevents nested metadata from masquerading as budget proof.
operations[].messageCharCount and providerUsage	Actual serialized message size and provider-reported usage when available.	Separates Glavred context size from provider-owned web-tool usage.
foundMaterialIds[]	Materials created by this run.	Links run trace to stored source material.
warnings[] and errors[]	Runtime and quality warnings/errors.	Prevents silent degradation.
benchmarkReport	Golden scenario verdict when available.	Gives a stable quality signal for matching runs.

The /radar-runs?runId=<id> page is a read-only view over this trace. It does not compute live quality in React.

Benchmark and Live Evaluation Contract

There are two benchmark modes:

- recorded: deterministic fixture-backed regression. It does not call providers.
- live: provider-backed run evaluated against golden expectations.

The live report must distinguish planned coverage from executed coverage:

- plannedCoverage: directions present in `searchPlan.intents[]`;
- executedCoverage: directions that produced executable queries, successful search operations, raw results, selected reads, or found materials;
- skippedRequiredCoverage: required directions that were planned but skipped or not actually executed, including reasons such as `budget-max-external-queries`.

Status vocabulary:

Status	Meaning
passed	Provider was usable, required coverage was actually executed, trace is complete, enough material/domain diversity exists, and known noise was not accepted.
warning	Provider was usable and useful material exists, but there are non-blocking gaps such as required planned directions skipped by budget, optional family gaps, provider degradation, or narrow read coverage.
failed	Provider was usable, but quality failed: required coverage/material/domain checks missed or accepted material contains known noise.
inconclusive	Provider/runtime state prevents a fair quality verdict: provider disabled, missing configuration, rate limit, network failure, or no honest execution trace.

passed means the search actually covered enough of the golden scenario. It does not mean only that the plan looked good.

Reading a RadarRun Trace

Use:

```
/radar-runs?runId=<RadarRun ID>
```

The trace should be read in this order:

1. Summary cards: status, budget, source coverage, material output, warnings/errors.
2. Campaign plan: strategy, language, source strategy, intent families, skipped intents.
3. Operations: provider query and URL-read statuses.
4. Search triage: result scores, duplicate groups, terminal decisions, read coverage, and gaps.
5. Raw results: bounded provider citations before triage.
6. Selected and rejected reads: what the run read and why every other result was rejected, duplicated, invalid, or deferred.
7. Found materials and read outcomes: what became readable upstream material and what remained metadata-only.

8. Benchmark report: whether the golden scenario verdict is passed, warning, failed, or inconclusive.
9. Raw JSON fallback when a legacy/minimal run lacks richer fields.

Known AS IS Limitations

These are current facts, not target architecture:

- Golden evaluation currently has one primary industrial AI scenario. More scenarios are needed before it becomes a broad quality gate.
- Search results are not yet stored in a reusable cross-run search memory. Rejected or unread results may be useful later, but current logic does not own that cache.
- The live query budget can skip required families such as limitation/critique. The report exposes this as warning-class coverage debt; the planner/budget tuning is a future product-quality slice.
- URL-read budget is intentionally narrow. A run may produce many raw results but read only a small subset.
- The current URL reader accepts text/HTML. Obvious PDF URLs are rejected before allocation; a binary or unsupported response is preserved only as metadata. A dedicated PDF/document reader is future adapter work.
- Glavred directly limits query text, provider input, serialized messages, and total local RadarRun input. OpenRouter may report much larger prompt-token usage because its web-search tool adds provider-owned retrieval context; production cost controls and reuse belong to Slice 2.17.4.6.6.
- Query family wording can still be too similar across broad discovery, case/example, and benchmark/paper directions.
- Signal extraction, signal scoring, and candidate assembly remain downstream and are not owned by RadarRun.
- The frontend trace page can display historical/minimal runs, but older runs may not contain all enriched fields.

Known limitations must be linked to roadmap slices rather than treated as successful target behavior.

Maintenance Rules

- Keep this document factual. Put future designs in TO BE documents or ADRs.
- If a RadarRun slice changes runtime order, trace shape, benchmark semantics, or output boundaries, update this Markdown and regenerate the PDF in the same slice.
- If a slice only changes code organization without changing runtime behavior, state that this AS IS remains valid in the final report.
- Do not use this document to bless hidden state, unbounded provider payloads, or downstream artifact creation inside RadarRun.